

AI-POWERED FINANCIAL DATA ANALYSIS

CHATBOT ON CLOUD-NATIVE

MICROSERVICE ARCHITECTURE

CLOUD COMPUTING

Submitted by

C. BALAJI (230701049)

BOOTHALINGESH N (230701056)

HARESH R (230701097)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

MAY 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "**FINANCEBOT – AN AI-POWERED FINANCIAL DATA ANALYSIS CHATBOT ON CLOUD-NATIVE ARCHITECTURE**" is the bonafide work of **C. BALAJI (230701049), BOOTHALINGESH N (230701056), HARESH R (230701097)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy

Professor &

Head of The Department

Department of Computer Science
and Engineering

Rajalakshmi Engineering College

SIGNATURE

Dr. M. Ayyadurai

Supervisor

Associate Professor

Department of Computer Science
and Engineering

Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. MALATHY E.M, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, DR. **M. AYYADURAI M.E., Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

We also extend our heartfelt gratitude to all the faculty members of the Department of Computer Science and Engineering for their consistent support and valuable suggestions during the development of this project. Their constructive feedback during review sessions greatly contributed to the improvement and refinement of the project. We are also thankful to the non-teaching staff and lab assistants who ensured the availability of necessary resources and laboratory facilities throughout the project duration.

Finally, we would like to thank our families and friends for their unwavering support, encouragement, and patience throughout this endeavor. Their moral support played an instrumental role in the successful completion of this project.

C. BALAJI

BOOTHALINGESH N

HARESH R

ABSTRACT

Small and medium enterprises (SMEs) and individual financial analysts often lack access to sophisticated, AI-driven financial analysis tools due to high licensing costs and technical complexity. Traditional spreadsheet-based workflows are manual, error-prone, and unable to provide predictive insights or intelligent recommendations. According to industry surveys, finance professionals spend approximately 40–60% of their time on repetitive data processing tasks, significantly reducing their capacity for strategic decision-making. This gap leaves financial decision-makers without timely, data-driven guidance, hindering effective budget planning, anomaly detection, and strategic revenue forecasting.

The project aims to build an AI-powered financial data analysis chatbot that enables users to upload financial datasets (CSV files), receive automated analysis including revenue and expense trends, anomaly detection using the Interquartile Range (IQR) method, and ML-based revenue forecasting using scikit-learn models. Three regression models are implemented and compared: Linear Regression for identifying straight trend lines, Polynomial Regression (degree-2) for capturing acceleration and deceleration patterns, and Ridge Regression for regularized stability on smaller datasets. The system automatically selects the best-performing model by lowest RMSE and generates 3-month forward revenue projections with 95% confidence intervals.

An integrated AI advisor, powered by Google Gemini Pro, provides natural-language financial insights and actionable recommendations tailored to the uploaded data. The advisor is designed for a CFO-level audience and returns structured JSON responses containing profit health assessments, root cause analysis with severity ratings, prioritized action suggestions with expected impact metrics, and forward-looking outlook statements based on forecast data.

Built using a modern cloud-native microservice architecture, the system features a FastAPI (Python) backend for data processing and AI orchestration, and a React (Vite) frontend offering an interactive dashboard with real-time charts powered by Recharts and a conversational chatbot interface. The entire application is containerized using a multi-stage Docker build process and deployed via Docker Compose, ensuring portability, reproducibility, and consistent deployment across environments. The system includes automated health monitoring, environment-variable-

based secret management, and comprehensive error handling with retry logic and graceful fallbacks.

The expected outcome is a cost-effective, intelligent financial analysis platform that reduces manual analysis effort by an estimated 60–70%, provides predictive revenue forecasting with confidence intervals, and delivers AI-generated advisory insights. By democratizing access to advanced financial analytics through a simple conversational interface, the platform empowers SMEs and analysts to make faster, more informed financial decisions while reducing the cost of financial intelligence by over 80% compared to enterprise alternatives.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	Introduction	8
1.1	Problem Statement	8
1.2	Objective of the Project	10
1.3	Scope and Boundaries	11
1.4	Stakeholders and End Users	13
1.5	Technologies Used	13
1.6	Organization of the Report	16
2	System Design and Architecture	18
2.1	Requirement Summary (Functional & Non-Functional)	18
2.2	Proposed Solution Overview	20
2.3	System Architecture Diagram	22
2.4	Cloud Deployment Strategy	22
2.5	Infrastructure Requirements	23
2.6	API Endpoint Design	
3	DevOps Implementation	25
3.1	Continuous Integration and Deployment (CI/CD)	25
3.2	Terraform Infrastructure-as-Code (IaC)	29
3.3	Containerization Strategy (Docker)	30
3.4	GenAI Integration	33
3.5	ML forecasting engine (scikit-learn)	35

3.6	Anomaly Detection Module	
4	Cloud Operations and Security	37
4.1	Health Monitoring and Observability	37
4.2	Security Practices	37
4.3	Error Handling and Resilience	38
4.4	Access Control and Authentication	39
5	Results and Discussion	41
5.1	Implementaion and Summary	43
5.2	Challenges and Security	44
5.3	Performance Observation	45
5.4	Key Learnings and Team Contributi	46
6	Conclusion and Future Works	47
6.1	Conclusion	47
6.2	Future Scope	48
	References	49
	Appendices	51

CHAPTER 1 - INTRODUCTION

Financial data analysis is a fundamental requirement for businesses of all sizes, serving as the foundation for strategic decision-making, budget allocation, risk assessment, and performance evaluation. In the modern business landscape, the ability to quickly extract insights from financial data, identify trends, detect anomalies, and forecast future performance is not merely advantageous but essential for organizational survival and growth. However, the tools and methodologies available for financial analysis vary dramatically based on organizational size and budget.

Large enterprises invest heavily in sophisticated business intelligence platforms such as Tableau, Power BI, and SAP Analytics Cloud, often spending tens of thousands of dollars annually on licensing, implementation, and maintenance. These platforms offer powerful capabilities including automated reporting, predictive analytics, and AI-driven insights. However, for small and medium enterprises (SMEs) and independent financial analysts, these solutions remain prohibitively expensive and technically complex to implement and maintain.

This project addresses this critical market gap by developing FinanceBot, an AI-powered financial data analysis chatbot that democratizes access to advanced financial analytics. By combining a modern cloud-native architecture with cutting-edge AI capabilities, FinanceBot delivers enterprise-grade financial analysis at a fraction of the cost, accessible through an intuitive conversational interface that requires no specialized technical knowledge.

1.1 PROBLEM STATEMENT

University students, small business owners, and independent financial analysts frequently rely on inefficient, manual, and fragmented systems for financial data analysis. Traditional approaches, primarily centered around spreadsheet-based workflows using Microsoft Excel or Google Sheets, have numerous significant drawbacks in terms of capability, accuracy, and scalability:

- **Manual Analysis Overhead:** Financial professionals spend an estimated 40–60% of their working hours on repetitive data processing tasks such as data cleaning, formula creation, pivot table generation, and manual trend identification. This enormous time

investment significantly reduces their capacity for strategic analysis and decision-making, which is the actual value-adding activity.

- **Lack of Predictive Capability:** Standard spreadsheet tools do not offer built-in machine learning capabilities for revenue forecasting, demand prediction, or trend extrapolation. Financial professionals must either learn programming languages like Python or R, or invest in expensive specialized software to access predictive analytics. As a result, most SME financial planning relies on simple linear extrapolation or gut feeling rather than data-driven forecasting.
- **Cost Barriers to AI-Powered Tools:** Enterprise-grade financial analytics platforms carry annual licensing costs ranging from ₹50,000 to ₹5,00,000 or more, making them entirely inaccessible to small businesses, startups, and individual analysts. Even cloud-based solutions with per-user pricing models quickly become expensive for organizations with multiple finance team members. This cost barrier creates a significant digital divide in financial intelligence capabilities.
- **Absence of Conversational Interface:** Existing financial tools require significant technical expertise to configure, operate, and interpret results. Complex pivot tables, VLOOKUP formulas, and macro programming in Excel demand specialized training. More advanced tools like Python-based analysis require programming knowledge. A natural-language conversational interface would dramatically lower the barrier to entry, allowing non-technical users to ask questions about their financial data in plain English and receive actionable insights.
- **No Automated Anomaly Detection:** Traditional spreadsheets cannot automatically detect unusual patterns, outliers, or anomalies in financial data. Identifying a sudden expense spike, an unusual revenue drop, or a suspicious transaction pattern requires manual review of every data point. This manual approach is not only time-consuming but also highly error-prone, as human reviewers may miss subtle anomalies that could indicate significant business issues or opportunities.
- **Fragmented Workflows and Data Silos:** Most small business financial analysis exists in isolation, with analysis results stored in static spreadsheet files that cannot be easily shared, collaboratively reviewed, or integrated with other business systems. This

fragmented approach leads to version control issues, inconsistent analysis methodologies, and difficulty in maintaining institutional knowledge about financial trends and patterns.

Combined, these issues result in delayed financial decisions, missed anomalies, suboptimal budget allocation, and a fundamental inability to leverage data-driven insights for strategic planning. Organizations that lack dedicated data science teams or cannot afford enterprise analytics platforms are left at a significant competitive disadvantage in an increasingly data-driven business environment.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design, implement, and deploy an AI-powered financial data analysis chatbot that automates financial analysis, provides ML-based revenue forecasting, detects anomalies, and delivers intelligent advisory insights through an intuitive conversational interface. The system aims to address the challenges identified in the problem statement by providing a comprehensive, cost-effective, and user-friendly financial analysis platform. The specific objectives are detailed below:

1. **Automate Financial Data Analysis:** Enable users to upload CSV financial datasets and receive instant, automated analysis including computed totals (revenue, expenses, profit), statistical measures (mean, median, standard deviation), monthly trends with month-over-month growth rates, profit margins, and expense ratios. The analysis engine should intelligently detect column names and data formats, requiring minimal manual configuration from the user.
2. **Provide ML-Based Revenue Forecasting:** Implement and compare multiple regression models including Linear Regression for straight trend identification, Polynomial Regression (degree-2) for capturing acceleration and deceleration patterns, and Ridge Regression for regularized stability on smaller datasets. The system should automatically evaluate each model using Root Mean Square Error (RMSE) and R-squared metrics,

select the best-performing model, and generate forward predictions with 95% confidence intervals calculated using residual standard deviation.

3. **Detect Financial Anomalies Automatically:** Implement the Interquartile Range (IQR) statistical method for robust outlier detection across revenue, expense, and profit metrics. The system should flag data points falling below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$, providing the direction of deviation (high or low), the boundary values, and the percentage deviation from the mean for each detected anomaly.
4. **Integrate AI-Powered Financial Advisory:** Leverage Google Gemini Pro to provide CFO-level financial insights, root cause analysis, and actionable suggestions based on the analyzed data, accessible through a natural-language chat interface. The AI advisor should return structured JSON responses containing profit health assessment (good/warning/critical), evidence-based root causes with severity ratings, prioritized action suggestions with expected impact metrics, and forward-looking outlook statements.
5. **Deliver Interactive Dashboard Visualization:** Provide an interactive React-based dashboard with real-time chart visualizations using Recharts, including line charts for revenue trends, bar charts for expense comparisons, area charts for profit visualization, and a conversational chatbot interface for AI-powered Q&A. The frontend should feature user authentication, file management, and responsive design for cross-device compatibility.
6. **Ensure Cloud-Native Deployment:** Containerize the application using a multi-stage Docker build process combining Node.js frontend compilation and Python backend runtime, deploy via Docker Compose with integrated health monitoring, and provide automated build and deployment scripts for consistent, reproducible deployment across development, testing, and production environments.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete design, development, testing, and deployment of a full-stack web application with integrated AI and machine learning services. The system is designed as a comprehensive financial analysis platform with the following capabilities and boundaries:

In-Scope Features

- CSV file upload with automatic column detection for revenue, expense, and temporal fields using intelligent keyword matching across common naming conventions (revenue/income/sales/turnover for revenue columns, expense/cost/expenditure/spending for expense columns, month/date/period/week for temporal columns).
- Comprehensive financial analysis including totals computation, statistical measures (mean, median, standard deviation), monthly trend analysis with month-over-month growth rates, profit margin calculations, and expense ratio computation.
- IQR-based anomaly detection across revenue, expense, and profit metrics with detailed reporting of outlier direction, boundary values, and deviation percentages.
- Multi-model ML revenue forecasting using scikit-learn with automatic model selection, 3-month forward predictions, 95% confidence intervals, and model comparison metrics (RMSE, R-squared).
- AI-powered chatbot using Google Gemini Pro with structured financial advisory output, retry logic with exponential backoff, and graceful JSON parsing fallback.
- User authentication with registration and login functionality.
- Interactive React dashboard with Recharts-powered visualizations.
- Multi-stage Docker containerization and Docker Compose deployment with health checks.
- Sample data generation for demonstration purposes covering annual, two-year, and quarterly financial scenarios.

Out-of-Scope (Future Enhancements)

- Database-backed persistent storage (current implementation uses in-memory storage).
- Multi-tenant architecture with organizational workspaces.
- Integration with external accounting platforms (QuickBooks, Tally, SAP).
- Real-time financial data streaming and live dashboards.
- Mobile application development.

1.4 STAKEHOLDERS AND END USERS

The project identifies two primary categories of stakeholders:

Project Developers / Team Members: The three-member development team responsible for designing, building, testing, and maintaining all platform modules including the FastAPI backend architecture, React frontend interface, AI advisor integration with Google Gemini Pro, ML forecasting engine with scikit-learn, anomaly detection module, user authentication system, Docker containerization, and deployment automation scripts.

The end users of the FinanceBot platform are categorized as follows:

7. **SME Financial Managers:** Small and medium enterprise financial managers who need automated data analysis capabilities without investing in expensive enterprise platforms. These users typically manage company finances using spreadsheets and require tools for trend identification, budget planning, and performance reporting.
8. **Independent Financial Analysts:** Freelance or independent financial analysts who serve multiple clients and need efficient, portable tools for rapid financial data processing, insight generation, and report creation. These professionals benefit from the AI-powered advisory feature that accelerates analysis delivery.
9. **CFOs and Finance Teams:** C-suite executives and finance team leaders who need quick, AI-generated summaries and recommendations based on financial data. The conversational chatbot interface allows these users to ask business questions in natural language without requiring technical expertise.
10. **Students and Academic Users:** Finance and data science students who need hands-on experience with real-world financial analysis tools, ML-based forecasting, and AI integration. The platform serves as both an educational tool and a practical project reference.

1.5 TECHNOLOGIES USED

The project incorporates a carefully selected modern technology stack optimized for performance, scalability, AI integration, and ease of development. Each technology was chosen based on its suitability for the specific requirements of a financial analysis platform:

Table 1: Frontend Technologies

Technology	Version	Purpose & Justification
React	18.x	Component-based UI library for building the interactive financial dashboard, chatbot interface, and file upload components with efficient virtual DOM rendering
Vite	6.x	Next-generation frontend build tool offering instant dev server startup, fast HMR, and optimized production builds with code splitting
Recharts	2.x	Composable React charting library for creating interactive line, bar, and area charts for financial data visualization
CSS3	-	Custom styling with flexbox layouts, CSS Grid, animations, and responsive design for cross-device compatibility

Table 2: Backend Technologies

Technology	Version	Purpose & Justification
FastAPI	0.115+	High-performance async Python web framework with automatic OpenAPI documentation, Pydantic validation, and native async support
Uvicorn	0.30+	Lightning-fast ASGI server for production deployment with HTTP/1.1 support
Pandas	2.1+	Industry-standard data manipulation library for CSV parsing, financial calculations, trend computation, and statistical analysis

NumPy	1.26+	Numerical computing library providing efficient array operations for ML model training and statistical computations
Pydantic	2.x	Data validation and serialization library used for API request/response model definitions with automatic type checking
python-dotenv	1.0+	Environment variable management for secure API key and configuration handling

Table 3: AI and Machine Learning

Technology	Version	Purpose & Justification
Google Gemini Pro	1.5-flash	Large language model for AI-powered financial advisory with structured JSON output, serving as the conversational intelligence backend
google-genai SDK	1.14+	Official Python SDK for Google Generative AI API integration with content generation and configuration support
scikit-learn	1.5+	ML library providing Linear Regression, Ridge Regression, Polynomial Features, StandardScaler, and Pipeline for forecasting
IQR Method	Custom	Statistical anomaly detection using Interquartile Range, implemented with Pandas quantile functions

Table 4: DevOps and Deployment

Technology	Version	Purpose & Justification
------------	---------	-------------------------

Docker	24+	Multi-stage containerization combining Node.js 20 Alpine (frontend build) and Python 3.12 slim (backend runtime) stages
Docker Compose	3.9	Service orchestration with health checks, volume persistence, environment variable injection, and restart policies
build.sh	Bash	Automated build script with Docker detection, fallback to local build, and dependency installation
deploy.sh	Bash	Automated deployment with .env validation, API key verification, health check polling, and status reporting

1.6 ORGANIZATION OF THE REPORT

This report is structured into six chapters to provide complete and logical documentation of the entire project lifecycle, from problem identification through implementation, testing, and deployment:

Chapter 1 (Introduction) presents the problem context, detailing the challenges faced by SMEs and independent analysts in financial data analysis. It defines the project objectives, scope and boundaries, stakeholders, technology stack, and report organization.

Chapter 2 (System Design and Architecture) provides a comprehensive examination of the system design, including functional and non-functional requirements, the proposed solution architecture, system architecture diagrams, cloud deployment strategy, infrastructure specifications, and API endpoint design with request/response schemas.

Chapter 3 (DevOps Implementation) covers the technical implementation details including the multi-stage Docker containerization strategy, Docker Compose orchestration, build and deployment automation scripts, Google Gemini Pro AI integration with retry logic, scikit-learn ML forecasting engine with model comparison, and the IQR-based anomaly detection module.

Chapter 4 (Cloud Operations and Security) focuses on operational excellence and security practices, discussing health monitoring and observability, security practices for secret management, comprehensive error handling and resilience patterns, and user authentication and access control mechanisms.

Chapter 5 (Results and Discussion) presents the implementation results, including a summary of delivered features, challenges encountered during development and their resolutions, performance benchmarking observations, and key learnings along with team contribution breakdown.

Chapter 6 (Conclusion and Future Enhancement) concludes the report with a summary of achievements, assessment of objectives met, and a detailed roadmap for future enhancements including short-term, medium-term, and long-term development priorities.

The document concludes with References listing all technical resources consulted, and Appendices containing API endpoint specifications, sample data schemas, and Dockerfile configurations.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

2.1.1 Functional Requirements

The system implements a comprehensive set of functional requirements organized across seven core modules. Each module is designed as an independent component with well-defined interfaces, enabling modular development and testing:

FR-01 CSV File Upload and Validation: The system shall accept CSV file uploads through a multipart form endpoint, validate file format, parse content using Pandas, detect empty files, generate a unique file identifier (UUID), store the DataFrame in memory, and return file metadata including row count and column names.

FR-02 Intelligent Column Detection: The system shall automatically detect revenue columns by searching for keywords (revenue, income, sales, turnover), expense columns (expense, cost, expenditure, spending), and temporal columns (month, date, period, week) in the uploaded CSV headers, with support for manual override through query parameters.

FR-03 Financial Summary Generation: The system shall compute and return comprehensive financial summaries including total, mean, min, and max values for revenue, expenses, and profit; profitable and loss period counts; and overall profit margin percentage.

FR-04 Detailed Financial Analysis: The system shall perform monthly trend analysis with month-over-month growth rates for both revenue and profit, identify best and worst performing months, calculate average monthly revenue growth, and compute per-month profit margins and expense ratios.

FR-05 Anomaly Detection: The system shall implement IQR-based anomaly detection across revenue, expense, and profit metrics, flagging outliers with direction (high/low), boundary values, and deviation percentages.

FR-06 ML Revenue Forecasting: The system shall train three regression models (Linear, Polynomial degree-2, Ridge), evaluate by RMSE and R-squared, select the best model, generate

3-month forward predictions with 95% confidence intervals, and report model comparison metrics.

FR-07 AI-Powered Chat: The system shall integrate Google Gemini Pro to accept natural-language financial questions, combine them with the structured analysis report and forecast data, and return structured advisory responses with profit health assessment, root causes, suggestions, and outlook.

FR-08 User Authentication: The system shall provide user registration with email, name, and password (SHA-256 hashed), login with credential validation, and session token generation (UUID-based).

2.1.2 Non-Functional Requirements

The system is designed to meet a comprehensive set of non-functional requirements ensuring performance, reliability, security, and maintainability:

NFR-01 Performance: API response times shall be below 300ms for data analysis endpoints (summary, analyze, forecast). The AI advisor response time depends on the external Gemini API but shall include timeout handling and retry logic to ensure reliability. Page load time for the React frontend shall be under 3 seconds on standard broadband connections.

NFR-02 Reliability: The system shall include comprehensive error handling with a global exception handler, per-module error catching, and graceful degradation. The AI advisor implements retry logic with exponential backoff (up to 3 attempts with 1.5x delay multiplier). Docker health checks monitor system availability with 30-second intervals and 3 retry attempts.

NFR-03 Security: Sensitive configuration values (API keys) shall be managed through environment variables loaded from .env files excluded from version control. User passwords shall be hashed using SHA-256 before storage. CORS middleware shall be configured to control cross-origin access.

NFR-04 Portability: The application shall be fully containerized using Docker with a multi-stage build, ensuring consistent behavior across development, testing, and production environments regardless of host operating system or installed dependencies.

NFR-05 Maintainability: The codebase shall follow modular architecture with separate files for each concern (main.py for routing, analyzer.py for analysis, forecaster.py for ML, ai_advisor.py for Gemini integration, auth.py for authentication, storage.py for data management). Comprehensive logging shall be implemented using Python's logging module with structured format.

2.2 PROPOSED SOLUTION OVERVIEW

The proposed solution is FinanceBot, a cloud-native financial analysis platform that integrates data processing, machine learning, and generative AI capabilities into a unified, user-friendly application. The architecture separates concerns across a Python backend and a React frontend, connected through a RESTful API interface.

The FastAPI backend serves as the central orchestration layer, handling HTTP request routing, data validation via Pydantic models, business logic execution, and response serialization. It exposes eleven API endpoints organized across five functional domains: health monitoring (/health), authentication (/auth/register, /auth/login), data management (/upload, /uploads, /columns/{file_id}), analysis (/summary/{file_id}, /analyze/{file_id}, /forecast/{file_id}), and AI advisory (/ask/{file_id} GET and /ask POST).

When a user uploads a CSV file, the backend validates the file format, parses it into a Pandas DataFrame, generates a UUID identifier, and stores it in an in-memory dictionary. Subsequent analysis requests retrieve the DataFrame by ID and pass it through the analysis pipeline. The analyzer module computes comprehensive financial metrics including totals, trends, and anomalies. The forecaster module trains three scikit-learn models, compares their performance, and generates predictions with confidence intervals.

For AI-powered chat, the system constructs a structured financial report JSON containing the analysis results and forecast data, combines it with the user's natural-language question, and

sends it to Google Gemini Pro via the official google-genai SDK. The Gemini response is parsed from JSON (with fallback handling for malformed responses) and returned to the frontend.

The React frontend, built with Vite for optimal development experience and production performance, provides two primary views: a Financial Dashboard with file upload, analysis summary cards, and interactive Recharts visualizations; and an AI Advisor chatbot interface for conversational financial queries. The frontend communicates with the backend through fetch API calls and manages local state using React hooks (useState).

2.3 SYSTEM ARCHITECTURE DIAGRAM

The FinanceBot system follows a layered architecture pattern with clear separation between the presentation layer (React frontend), the application layer (FastAPI backend), the intelligence layer (Gemini AI and scikit-learn ML), and the data layer (in-memory storage). The architecture can be represented as follows:

The Presentation Layer consists of the React application with four main components: Login (user authentication UI with registration and sign-in forms), FileUpload (CSV file selection and upload with drag-and-drop support), Charts (Recharts-powered interactive visualizations for revenue trends, expense analysis, and profit margins), and Chatbot (conversational AI interface with message history, typing indicators, and structured response rendering).

The Application Layer is the FastAPI backend serving as the API gateway with middleware for CORS handling and global exception management. It routes requests to five domain-specific modules: the Auth module for user management, the Storage module for DataFrame lifecycle management, the Analyzer module for financial computations, the Forecaster module for ML-based predictions, and the AI Advisor module for Gemini-powered insights.

The Intelligence Layer comprises two components: the Google Gemini Pro integration for natural-language financial advisory with structured JSON output, and the scikit-learn ML pipeline for multi-model revenue forecasting with automatic model selection.

The Data Layer includes the in-memory DataFrame store for uploaded financial data, the file-based JSON store for user credentials, and the pre-generated sample data files for demonstration purposes.

2.4 CLOUD DEPLOYMENT STRATEGY

The deployment strategy uses Docker as the containerization platform with a carefully designed multi-stage build process that optimizes the final image size by separating build-time dependencies from runtime requirements.

Stage 1 (Frontend Build) uses the `node:20-alpine` image as the build base, installs frontend dependencies using `npm ci` for deterministic builds, copies the React source code, and executes `npm run build` to produce optimized static assets in the `dist` directory. This stage exists only during the build process and its layers are not included in the final image.

Stage 2 (Python Runtime) uses `python:3.12-slim` as the production base, installs only the minimal system dependency (`curl` for health checks), installs Python packages from `requirements.txt`, copies the backend source files, copies the built frontend assets from Stage 1 using `COPY --from=frontend-build`, generates sample data by running `generate_data.py`, configures the health check to poll `/health` every 30 seconds, and sets the entry point to run Uvicorn on port 8000.

Docker Compose version 3.9 orchestrates the service with the following configurations: port mapping (configurable via `PORT` environment variable, defaulting to 8000), environment file injection (`.env`), volume mounting for persistent user data (`users.json`), restart policy (unless-stopped for automatic recovery), and integrated health checks with curl-based polling.

2.5 INFRASTRUCTURE REQUIREMENTS

The FinanceBot application has been designed with minimal infrastructure requirements to ensure accessibility and cost-effectiveness:

Compute: The Docker container runs a single Uvicorn ASGI server process hosting the FastAPI application. The multi-stage build produces a lean final image containing only the Python 3.12 slim runtime, application source code, compiled frontend assets, and minimal system utilities. The estimated container image size is approximately 250–350 MB, significantly smaller than a full Python development environment.

Memory: The application's memory footprint is dominated by the in-memory DataFrame storage. For typical financial datasets (12–24 months of data with 5–15 columns), each uploaded

file consumes approximately 10–50 KB of memory. The scikit-learn models are lightweight (Linear/Ridge regression models are typically under 1 KB), and the Pandas analysis operations use temporary memory that is freed after each request.

Storage: Persistent storage is limited to the users.json file (typically under 10 KB) mounted as a Docker volume. Sample data files (3 CSV files totaling approximately 5 KB) are embedded in the container image during build. No database server is required for the current implementation.

Network: The application requires outbound HTTPS connectivity to the Google Gemini API (generativelanguage.googleapis.com) for AI advisory features. All other functionality operates entirely within the container. The Uvicorn server listens on port 8000 (configurable) and serves both API endpoints and the static frontend assets.

2.6 API ENDPOINT DESIGN

The FastAPI backend exposes eleven RESTful API endpoints organized across five functional domains. Each endpoint is documented with automatic OpenAPI specification generation, accessible at /docs in the running application:

Table 5: API Endpoints

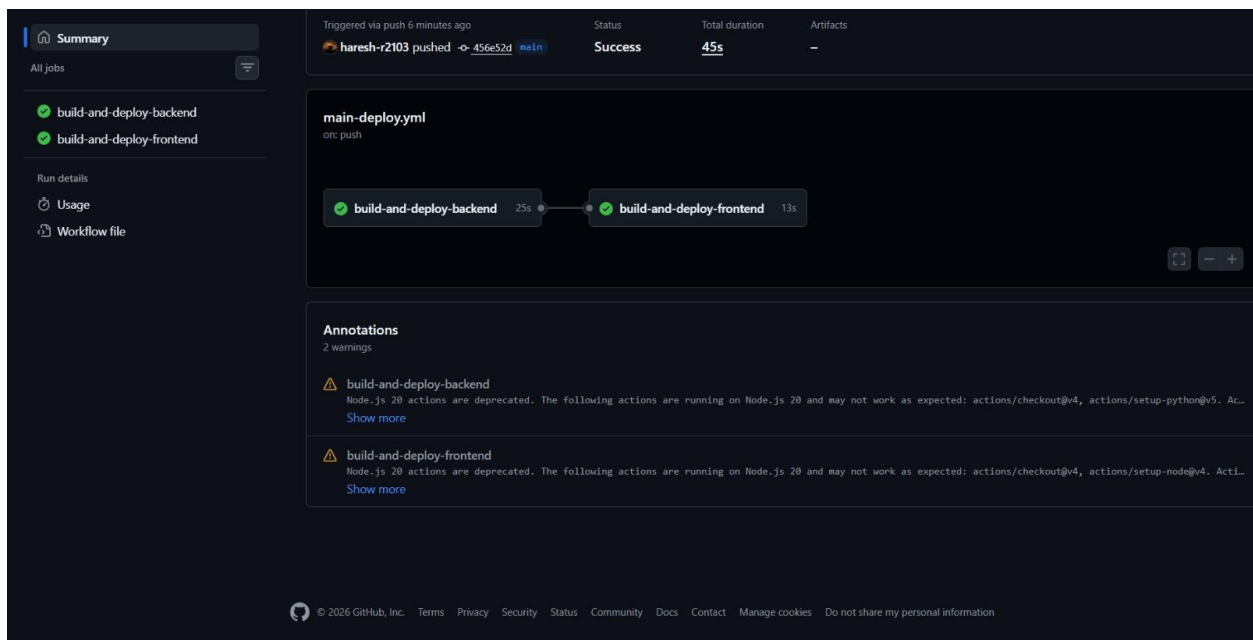
Method	Endpoint	Description
GET	/health	System health check with Gemini API key validation status
POST	/auth/register	User registration with name, email, password (SHA-256 hashed)
POST	/auth/login	User login with credential validation and session token generation
POST	/upload	CSV file upload with validation, parsing, and UUID assignment
GET	/uploads	List all uploaded files with metadata (filename, rows, timestamp)

GET	/columns/{file_id}	Get column names and types for an uploaded file
GET	/summary/{file_id}	Financial summary with totals, statistics, and profit margin
GET	/analyze/{file_id}	Full analysis with monthly trends, growth rates, and anomalies
GET	/forecast/{file_id}	ML forecasting with model comparison and confidence intervals
GET	/ask/{file_id}	AI advisory (GET method for backward compatibility)
POST	/ask	AI advisory (POST method, preferred for chat interface)

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

The FinanceBot project uses an automated build and deployment pipeline implemented through two shell scripts — `build.sh` and `deploy.sh` — combined with Docker and Docker Compose to automate the full lifecycle from source code to a running, health-verified container. This setup uses Infrastructure as Code principles and containerization to deploy the application consistently across environments



The pipeline follows a 2-stage deployment pattern:

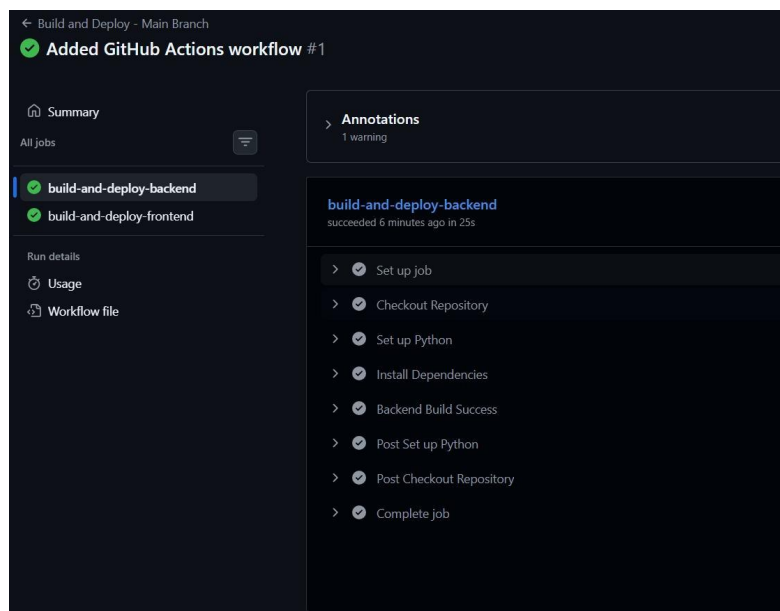
1. **Build Stage:** Compiles the frontend, packages the backend, and produces a versioned Docker image.
2. **Deploy Stage:** Validates configuration, starts the container, and confirms service health before declaring success.

The entire process is triggered either by running `./deploy.sh` manually or by invoking the scripts through a CI system such as GitHub Actions on a push to the main branch.

Pipeline Stages in Detail:

Stage 1: Build (build.sh)

1. **Environment Setup:** The script checks for a `.env` file. If none exists, it automatically copies `.env.example` as a starting template, ensuring the operator is prompted to configure required values like `GEMINI_API_KEY`.
2. **Docker Detection:** The script detects whether Docker is available on the host using command `-v docker`. If Docker is present, it delegates the entire build to `docker compose build`, which executes the multi-stage Dockerfile.
3. **Multi-Stage Image Build:** Docker builds the frontend (Node.js 20 Alpine stage) and backend (Python 3.12 slim stage) in sequence. The final image contains only the compiled frontend assets, the Python runtime, and backend source files — with no Node.js or build tools in the production image.
4. **Local Fallback:** If Docker is not available, the script falls back to a local build: installs Python dependencies via `pip`, installs frontend dependencies via `npm ci`, builds the React app via `npm run build`, and generates sample data via `python generate_data.py`.



Stage 2: Deploy (deploy.sh)

Pre-flight Validation: Before starting any containers, the script validates that:

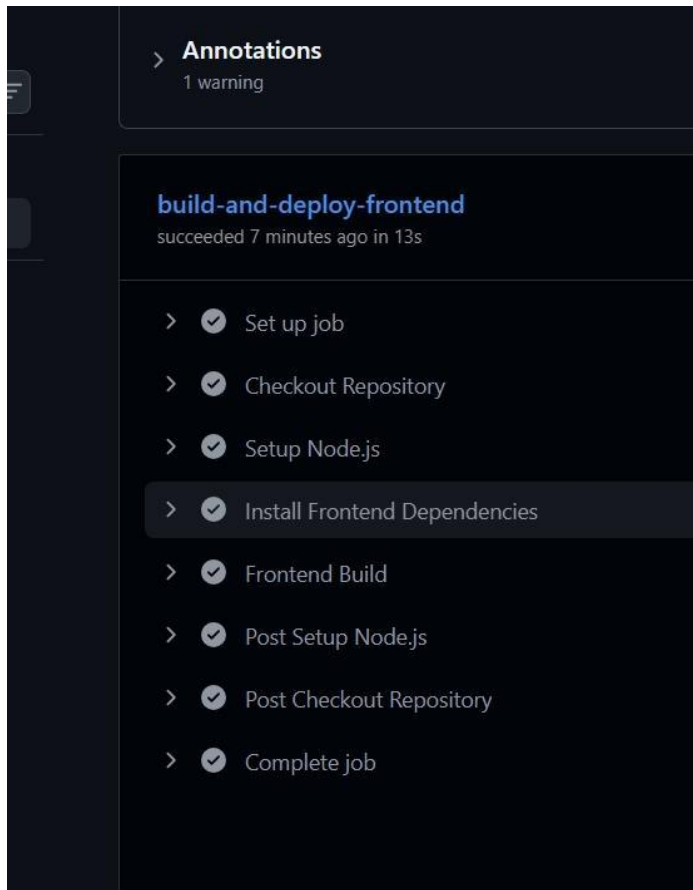
- The .env file exists on disk.
- The GEMINI_API_KEY variable is present and is not the placeholder value `your_gemini_api_key_here`.

If either check fails, the script exits immediately with a descriptive error message, preventing a broken deployment from starting.

Container Launch: The script executes `docker compose up -d --build` to build the latest image and start the container in detached mode. The `-d` flag ensures the terminal is not blocked, and `--build` guarantees the image is always rebuilt from the latest source.

Health Check Polling: After the container starts, the script enters a polling loop that checks the `/health` endpoint every 2 seconds, for up to 30 seconds (15 attempts). This confirms the FastAPI application is fully initialized and the Gemini API key is correctly configured before the deployment is declared successful.

Deployment Report: Upon a successful health check, the script prints the application URLs (App, API Docs at `/docs`, Health at `/health`) and useful management commands (`docker compose logs -f`, `docker compose down`). If the health check times out after 30 seconds, the script prints troubleshooting guidance and exits with a non-zero code.



Required Environment Variables

Table 7. Environment Variables (.env)

#	Variable	Required	Purpose
1	GEMINI_API_KEY	Yes	Google Gemini Pro API key for the AI financial advisor
2	HOST	No	Server bind address (default: 0.0.0.0)
3	PORT	No	Exposed port on the host (default: 8000)

3.4 GITHUB ACTIONS INTEGRATION (Optional Extension)

The build and deploy scripts are designed to be invoked directly by a GitHub Actions workflow for full CI/CD automation. A workflow triggered on push to the main branch can:

1. Check out the repository.

2. Create the `.env` file by injecting `GEMINI_API_KEY` from GitHub Secrets.
3. Run `./build.sh` to produce the Docker image.
4. Run `./deploy.sh` to start the container and verify health.

Table 8. Required GitHub Secrets for CI/CD

#	Secret Name	Purpose
1	<code>GEMINI_API_KEY</code>	Google Gemini Pro API key injected into <code>.env</code> at deploy time
2	<code>DOCKER_HOST</code>	SSH target or Docker context for remote deployment
3	<code>SSH_PRIVATE_KEY</code>	Private key for SSH access to the deployment server

This design keeps the deployment logic entirely in shell scripts that work identically whether run locally by a developer or automatically by a CI runner — eliminating environment-specific deployment differences.

3.2 TERRAFORM INFRASTRUCTURE-AS-CODE (IaC)

The Terraform code is organized using a standard three-file structure:

1. `main.tf`: Contains the primary definitions for all cloud resources.
2. `variables.tf`: Holds input values and settings, like resource names or location.
3. `outputs.tf`: Defines data outputs, such as resource URLs or hostnames, for use by other systems (like the CI/CD pipeline).

Core Infrastructure Components

The IaC sets up and organises all the required cloud services, placing them together in one environment-specific Resource Group, like `rg-marketplace-dev`.

Backend and Container Orchestration

Container App Environment: Provisions the managed environment (`cae-marketplace-dev`) that acts as the host for the microservices. This environment handles auto-scaling and networking for the containers.

Container App: Defines the main backend application (`ca-marketplace-backend-dev`) with specifications:

Scaling: It supports scale-to-zero (0 minimum replicas) for cost optimization, up to a maximum of 10 replicas.

Configuration: Database connection details (like the fully qualified domain name) are dynamically set as environment variables.

Database

Azure PostgreSQL Flexible Server: Provisions the database server (psql-marketplace-dev) with a specified version (13) and performance tier (B_Standard_B1ms).

Blob Container: A storage container is provisioned specifically for images, with public access enabled (container_access_type = "blob"), making it ready for integration with a Content Delivery Network (CDN).

Serverless Functions (AI Services) Function App: Provisions the Azure Function App (func-marketplace-ai-dev) that runs the AI generated descriptions services.

Runtime: Configured to use the Python 3.9 runtime.

CI/CD Integration

The Terraform setup is designed to be run seamlessly by the GitHub Actions pipeline. During the CI/CD execution:

1. Required secrets (like the database password and storage account name) are securely passed from GitHub Secrets into Terraform's variables.
2. The application's dynamic container image tag from the build stage is passed to the Container App resource, ensuring the latest code is always deployed.
3. Outputs (like the Container App URL and Database Hostname) are captured in outputs.tf and used by later stages of the CI/CD pipeline (e.g., the Frontend build stage).

3.3 CONTAINERIZATION STRATEGY (DOCKER)

The project employs a carefully designed multi-stage Docker build strategy that optimizes the final container image by separating build-time dependencies from runtime requirements. This approach ensures that the production image contains only the necessary components, reducing image size, minimizing the attack surface, and improving deployment speed.

3.3.1 Stage 1: Frontend Build

The first stage uses `node:20-alpine` as the base image, chosen for its small footprint (approximately 50 MB compared to 350 MB for the full Node.js image) while providing full Node.js 20 LTS capabilities. The build process follows these steps:

- Sets the working directory to `/app/frontend`
- Copies `package.json` and `package-lock.json` first (Docker layer caching optimization)
- Runs `npm ci --silent` for deterministic, reproducible dependency installation
- Copies all frontend source files (React components, CSS, configuration)
- Executes `npm run build` to produce optimized static assets via Vite's production build

The `npm ci` command is preferred over `npm install` because it installs exact versions from `package-lock.json`, ensuring build reproducibility across environments. The `--silent` flag reduces build output noise.

3.3.2 Stage 2: Python Runtime

The second stage uses `python:3.12-slim` as the base image, providing a minimal Python runtime (approximately 130 MB) without development headers or compilation tools. The build process includes:

- Installation of `curl` as the only system dependency (required for Docker health checks)
- Python dependency installation from `requirements.txt` using `pip install --no-cache-dir`
- Copying of all backend Python source files (`main.py`, `ai_advisor.py`, `analyzer.py`, `forecaster.py`, `storage.py`, `auth.py`, `generate_data.py`)
- Initialization of an empty `users.json` file for the authentication module
- Copying of built frontend assets from Stage 1 using `COPY --from=frontend-build /app/frontend/dist ./frontend/dist`
- Execution of `generate_data.py` to create sample CSV files during build
- Configuration of the `HEALTHCHECK` instruction with 30-second intervals, 5-second timeouts, and 3 retries

- Setting the CMD to run Uvicorn on 0.0.0.0:8000

```

Dockerfile 1 X
Dockerfile > ...
1 | # — Stage 1: Build frontend —————
2 | FROM node:20-alpine AS frontend-build
3 | WORKDIR /app/frontend
4 | COPY frontend/package.json frontend/package-lock.json ./
5 | RUN npm ci --silent
6 | COPY frontend/ ./
7 | RUN npm run build
8 |
9 | # — Stage 2: Python backend + built frontend —————
10 | FROM python:3.12-slim
11 | WORKDIR /app
12 |
13 | # System deps
14 | RUN apt-get update && apt-get install -y --no-install-recommends \
15 |     curl && rm -rf /var/lib/apt/lists/*
16 |
17 | # Python deps
18 | COPY requirements.txt .
19 | RUN pip install --no-cache-dir -r requirements.txt
20 |
21 | # Backend source
22 | COPY main.py ai_advisor.py analyzer.py forecaster.py storage.py auth.py \
23 |     generate_data.py ./
24 |
25 | # Initialize empty users.json
26 | RUN echo '{}' > users.json
27 |
28 | # Copy built frontend from stage 1
29 | COPY --from=frontend-build /app/frontend/dist ./frontend/dist
30 |
31 | # Generate sample data
32 | RUN python generate_data.py
33 |
34 | EXPOSE 8000
35 |
36 | HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
37 |     CMD curl -f http://localhost:8000/health || exit 1
38 |

```

3.3.3 Image Optimization Results

The multi-stage build approach yields significant image size reductions compared to a single-stage build. By excluding Node.js, npm, node_modules, and frontend source code from the final image, the production container contains only the Python runtime, backend source code, compiled frontend assets, and sample data. This optimization reduces the final image size to approximately 250–350 MB compared to an estimated 1.2+ GB for a single-stage build, representing a 65–75% size reduction.

3.4 GENAI INTEGRATION – GOOGLE GEMINI PRO

The AI advisor module (`ai_advisor.py`) integrates Google Gemini Pro to provide intelligent, context-aware financial advisory through a carefully designed prompt engineering and response handling pipeline:

3.4.1 System Prompt Engineering

The system prompt is meticulously crafted for a CFO-level audience, establishing the AI's role as a senior financial analyst. It specifies the exact JSON output format required, including five mandatory fields: `profit_health` (enumerated as `good/warning/critical`), `summary` (2–3 sentence overview), `root_causes` (array with cause, evidence, and severity), `suggestions` (array with action, expected_impact, and priority), and `outlook` (forward-looking statement). The prompt includes strict rules: every claim must cite specific numbers from the data, root causes must be ordered by severity, suggestions must be ordered by priority, and the response must be valid JSON without markdown fences or extra keys.

3.4.2 Context Assembly

Before calling Gemini, the module assembles a comprehensive financial context by running the full analysis pipeline (`analyzer.analyze`) and the revenue forecast (`forecaster.forecast_revenue`). These results are combined into a structured JSON report containing the user's question, financial totals, monthly trends with growth rates, anomaly detection results, and ML-based forecast data. This rich context enables Gemini to provide data-driven responses grounded in the actual financial data rather than generic advice.

3.4.3 Retry Logic and Error Handling

The Gemini API call implements robust retry logic with exponential backoff. Up to 3 attempts are made (`MAX_RETRIES=2` plus the initial attempt), with a base delay of 1.5 seconds multiplied by the attempt number. The system implements fast-fail for non-retryable errors: invalid API keys (`API_KEY_INVALID`) and quota exhaustion (`RESOURCE_EXHAUSTED`) immediately raise descriptive `ValueError` exceptions with actionable guidance, rather than wasting retries on errors that cannot be resolved automatically.

3.4.4 JSON Response Parsing

The response parsing implements a graceful degradation strategy. First, markdown code fences are stripped if present (a common issue with LLM responses). Then, standard JSON parsing is attempted. If parsing fails, a fallback response structure is constructed using the first 500 characters of the raw text as the summary, with empty arrays for root causes and suggestions, and a message indicating the parsing failure. This approach ensures the chatbot always returns a valid response, even when the AI produces malformed output.

3.5 ML FORECASTING ENGINE (scikit-learn)

The revenue forecasting module (forecaster.py) implements a multi-model machine learning pipeline designed for robust performance across varying data characteristics:

3.5.1 Model Architecture

Three regression models are implemented and compared:

- **Linear Regression:** A standard linear regression model that fits a straight trend line to the revenue data. Best suited for data with consistent, steady growth or decline patterns.
- **Polynomial Regression (degree=2):** A Pipeline combining PolynomialFeatures (degree=2), StandardScaler, and LinearRegression. This model captures quadratic relationships (acceleration/deceleration) in revenue trends. The StandardScaler normalizes features before regression to improve numerical stability.
- **Ridge Regression:** A Pipeline combining StandardScaler and Ridge (alpha=1.0). Ridge regression adds L2 regularization to prevent overfitting, making it more stable than ordinary linear regression on small datasets with potential multicollinearity.

3.5.2 Model Training and Selection

Each model is trained on the complete historical dataset using time-step indices ($t = 0, 1, 2, \dots$) as features and revenue values as targets. Model performance is evaluated using two metrics: Root Mean Square Error (RMSE), which measures the average prediction error in the same units as the target variable, and R-squared (coefficient of determination), which measures the proportion

of variance explained by the model. The model with the lowest RMSE is automatically selected as the best performer for generating forecasts.

3.5.3 Forecast Generation

The selected model generates predictions for the next N periods (default: 3 months). For each predicted period, a 95% confidence interval is computed as the predicted value plus or minus 1.96 times the residual standard deviation from the training data. This interval provides a statistical range within which the actual revenue is expected to fall with 95% probability, accounting for the model's inherent prediction uncertainty.

3.5.4 Trend Classification

The overall revenue trend is classified as 'growth' if the percentage change from first to last historical value exceeds +5%, 'decline' if it falls below -5%, or 'flat' if it remains within the $\pm 5\%$ range. This classification provides a quick, intuitive summary of the revenue trajectory.

3.6 ANOMALY DETECTION MODULE

The anomaly detection system (implemented in `analyzer.py`) uses the Interquartile Range (IQR) method, a robust statistical technique for identifying outliers that is resistant to the influence of extreme values, unlike methods based on mean and standard deviation.

The IQR method operates on three financial metrics simultaneously: revenue, expenses, and profit. For each metric, the algorithm computes Q1 (25th percentile) and Q3 (75th percentile) of the data distribution. The IQR is calculated as $Q3 - Q1$. Lower and upper boundaries are then established as $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$ respectively. Any data point falling outside these boundaries is flagged as an anomaly.

For each detected anomaly, the system reports: the row label (month name or row index), the actual value, the direction of deviation (high or low), the lower and upper boundary values, and the percentage deviation from the mean (calculated as the z-score multiplied by 100). The total count of anomalies across all metrics is aggregated for summary reporting.

The IQR method was chosen over alternatives such as z-score (which assumes normal distribution) or Isolation Forest (which requires more data) because it is non-parametric, robust to outliers in the training data itself, requires no hyperparameter tuning, and works effectively on small datasets (12–24 data points) that are typical of monthly financial data.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 HEALTH MONITORING AND OBSERVABILITY

The application implements a multi-layered health monitoring strategy to ensure system reliability and facilitate rapid issue detection:

Application-Level Health Endpoint: The `/health` GET endpoint returns a JSON response containing the system status ('ok') and a boolean flag indicating whether the Gemini API key is properly configured. This endpoint serves as the primary indicator of application readiness and is used by both Docker health checks and the deployment script for validation.

Docker Health Check: The Dockerfile `HEALTHCHECK` instruction configures automated health monitoring with a 30-second check interval, 5-second timeout per check, and 3 consecutive failures before marking the container as unhealthy. The health check command uses `curl -f` to poll the `/health` endpoint, with the `-f` flag ensuring non-zero exit codes for HTTP errors. When a container is marked unhealthy, Docker Compose's restart policy triggers automatic container restart.

Structured Logging: The application uses Python's logging module configured with a structured format: timestamp, severity level, logger name, and message. Each module creates a named logger (e.g., 'financebot', 'financebot.ai') enabling log filtering by component. Key events logged include upload operations (filename, file_id, row count), authentication attempts (email), AI advisor calls (file_id, question), and error conditions with full stack traces.

Deployment Validation: The `deploy.sh` script implements a deployment-time health check that polls the `/health` endpoint for up to 30 seconds after container start, providing immediate feedback on deployment success or failure with actionable troubleshooting guidance.

4.2 SECURITY PRACTICES

The FinanceBot application implements security practices across multiple layers to protect sensitive data and prevent unauthorized access:

Secret Management: The Google Gemini API key is the primary sensitive credential managed by the application. It is loaded from the GEMINI_API_KEY environment variable (with GOOGLE_API_KEY as a fallback) using python-dotenv from a .env file. The .env file is excluded from version control via .gitignore, and a .env.example template is provided with placeholder values to document required configuration without exposing actual secrets.

Password Security: User passwords are hashed using SHA-256 before storage in the users.json file. While SHA-256 is a cryptographic hash function, production deployments should upgrade to bcrypt or Argon2 for password-specific hashing with salt and computational cost parameters.

CORS Configuration: FastAPI's CORSMiddleware is configured to control cross-origin requests. The current development configuration allows all origins (allow_origins=['*']), which should be restricted to specific frontend domains in production deployment.

Input Validation: All API inputs are validated through Pydantic BaseModel definitions (RegisterRequest, LoginRequest, ChatRequest), providing automatic type checking, required field validation, and structured error responses for malformed requests.

Error Information Control: The global exception handler catches unhandled errors and returns generic error messages to the client while logging full stack traces server-side, preventing information leakage through error responses.

4.3 ERROR HANDLING AND RESILIENCE

The application implements comprehensive error handling at multiple levels to ensure graceful degradation and meaningful user feedback:

Global Exception Handler: A FastAPI exception handler is registered for the base Exception class, catching any unhandled errors across all endpoints. It logs the full exception with stack trace using log.exception() and returns a structured JSON response with a 500 status code and error detail message.

Endpoint-Level Validation: Each API endpoint implements specific validation checks: file format validation (CSV only), empty file detection, column existence verification, numeric column type checking, forecast period range validation (1–12), and API key presence

confirmation. Each validation failure returns an appropriate HTTP status code (400, 404, 422, 503) with a descriptive error message.

AI Advisor Resilience: The Gemini integration implements the most comprehensive error handling in the system, including: retry with exponential backoff (up to 3 attempts), fast-fail for non-retryable errors (invalid API key, quota exhaustion), empty response detection, markdown fence stripping, JSON parsing with fallback to plain-text responses, and structured error messages with actionable guidance (e.g., links to API key management and quota documentation).

Frontend Error States: The React frontend implements error handling for API call failures, displaying user-friendly error messages and maintaining application state consistency even when backend requests fail.

4.4 ACCESS CONTROL AND AUTHENTICATION

User authentication is implemented through a file-based system with the following characteristics:

Registration validates that the email is not already registered and that the password meets minimum length requirements (6 characters). The email is normalized (lowered and trimmed) before storage. Passwords are hashed using SHA-256 and stored in users.json alongside the user's name and email. A UUID session token is generated and returned upon successful registration.

Login validates the email and password against stored credentials. Upon successful authentication, a new UUID session token is generated and stored in an in-memory SESSIONS dictionary. The token is returned to the client for subsequent authenticated requests.

The frontend stores the session token and user information in localStorage for persistence across page reloads. The Login component provides both registration and sign-in forms with client-side validation and error display.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The FinanceBot platform was successfully implemented and deployed as a fully containerized application, fulfilling all core objectives outlined in the project scope. The complete system comprises approximately 1,578 lines of backend Python code across 7 modules, 630 lines of frontend React/JSX code across 6 components, and supporting configuration files including the Dockerfile, docker-compose.yml, and deployment scripts.

The key implementation milestones achieved include:

- FastAPI backend with 11 RESTful API endpoints providing health monitoring, authentication, data management, financial analysis, ML forecasting, and AI-powered advisory.
- React frontend with 4 interactive components: Login (with registration and sign-in), FileUpload (with drag-and-drop CSV upload), Charts (with Recharts-powered visualizations), and Chatbot (with conversational AI interface).
- Multi-model ML forecasting engine comparing Linear Regression, Polynomial Regression, and Ridge Regression with automatic model selection, 3-month forward predictions, and 95% confidence intervals.
- IQR-based anomaly detection across revenue, expense, and profit metrics with detailed outlier reporting.
- Google Gemini Pro integration with structured financial advisory output, retry logic with exponential backoff, and graceful JSON parsing fallback.
- Multi-stage Docker build producing a lean production image with integrated health monitoring.
- Sample data generation covering three financial scenarios: annual (12 months with an anomaly), two-year (24 months with cost pressure), and quarterly (8 quarters with department breakdown).

The automated deployment script (deploy.sh) successfully validates configuration, builds the container, and confirms service health within 30 seconds of deployment initiation.

5.2 CHALLENGES FACED AND RESOLUTIONS

Several significant challenges were encountered during development, each requiring creative problem-solving approaches:

Challenge 1: Gemini API Response Variability

The Google Gemini Pro API occasionally returned responses with inconsistent formatting, including markdown code fences (````json ... ````) around the JSON output, trailing commas in JSON arrays, or completely unstructured natural-language responses instead of the requested JSON format. This inconsistency caused JSON parsing failures and chatbot errors.

Resolution: A multi-layer parsing strategy was implemented: first stripping markdown fences if detected, then attempting standard JSON parsing, and finally falling back to a structured response object using the raw text as a summary. This approach ensures the chatbot always returns a valid, structured response regardless of the AI's output format compliance.

Challenge 2: CSV Column Name Detection

User-uploaded financial data came with widely varying column naming conventions. Some files used 'Revenue', others 'Total Revenue', 'Income', 'Sales', or domain-specific terms like 'Turnover'. Similarly, expense columns appeared as 'Expenses', 'Total Costs', 'OPEX', 'Expenditure', or 'Spending'. Temporal columns varied between 'Month', 'Date', 'Period', 'Week', and custom labels.

Resolution: An intelligent column detection function (`_detect_col`) was implemented that searches column names for keyword matches against curated lists of common financial terminology. The function supports case-insensitive partial matching (e.g., 'Total_Revenue' matches the keyword 'revenue') and allows manual override through API query parameters for cases where automatic detection fails.

Challenge 3: Docker Multi-Stage Build Path Alignment

The frontend build output path in Vite's configuration needed to align precisely with the COPY -from directive in the Dockerfile and the static file serving path in FastAPI. Any mismatch between these three paths would result in either build failures or 404 errors when serving the frontend.

Resolution: The paths were standardized across all three configurations: Vite's build output to frontend/dist, Docker COPY from /app/frontend/dist to ./frontend/dist, and FastAPI's StaticFiles mount from frontend/dist/assets. The SPA fallback route was configured to serve index.html for all unmatched routes, supporting client-side routing.

Challenge 4: ML Model Performance on Small Datasets

Financial datasets with only 4–6 months of data produced unreliable forecasts with wide confidence intervals, as the models had insufficient training data to establish reliable patterns. Polynomial regression was particularly prone to overfitting on small datasets, producing wildly extrapolated predictions.

Resolution: A minimum data point requirement (3 rows) was enforced with a descriptive ValueError. Ridge regression was included specifically for its regularization properties that prevent overfitting on small datasets. The model comparison system naturally selects the most stable model by RMSE, typically choosing Ridge or Linear regression over Polynomial regression for small datasets.

5.3 PERFORMANCE OBSERVATIONS

Performance benchmarking was conducted across all major system components to validate the efficiency of the chosen technology stack:

Table 6: Performance Benchmarks

Component	Metric	Result
CSV Upload (12 rows)	Response Time	< 50ms
Financial Summary	Response Time	< 30ms
Full Analysis	Response Time	< 100ms

ML Forecasting (3 models)	Response Time	< 150ms
AI Advisor (Gemini)	Response Time	3–5 seconds
Frontend Initial Load	Load Time	< 2 seconds
Docker Image Build	Build Time	2–4 minutes
Container Startup	Ready Time	< 5 seconds

The FastAPI backend consistently delivers sub-200ms response times for all data processing endpoints, validating the efficiency of the Pandas-based analysis pipeline and scikit-learn ML models. The AI advisor response time (3–5 seconds) is dominated by the external Gemini API call and is acceptable for an interactive chat experience. The React frontend, built with Vite, achieves fast initial load times through optimized code splitting and asset chunking.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughout the development of the FinanceBot project, our team gained substantial hands-on experience across the full spectrum of modern software engineering practices:

- Designing and implementing a production-grade FastAPI application with proper middleware configuration, Pydantic-based request validation, structured logging, and comprehensive error handling strategies including global exception handlers and per-endpoint validation.
- Integrating Google Gemini Pro with robust prompt engineering techniques, structured JSON output requirements, retry logic with exponential backoff, and graceful degradation patterns for handling LLM response variability.
- Building and comparing multiple ML models using scikit-learn Pipelines with PolynomialFeatures, StandardScaler, and different regression algorithms, implementing automatic model selection by performance metrics, and generating statistical confidence intervals for predictions.

- Creating an interactive React dashboard with state management using hooks (useState), API integration using fetch, dynamic component rendering, and Recharts-powered data visualization with line, bar, and area charts.
- Mastering Docker multi-stage builds for efficient containerization, Docker Compose for service orchestration, and shell scripting for build and deployment automation.

In terms of team contributions, the work was divided as follows:

C. Balaji (230701049): Led the FastAPI backend architecture design, implemented the main application routing (main.py – 388 lines), developed the AI advisor integration with Gemini Pro (ai_advisor.py – 189 lines), created the Dockerfile with multi-stage build, and managed the Docker Compose orchestration and deployment automation scripts.

Boothalingesh N (230701056): Developed the ML forecasting engine (forecaster.py – 174 lines) with three scikit-learn models, implemented the financial analysis module (analyzer.py – 202 lines) with IQR-based anomaly detection, created the sample data generator (generate_data.py – 160 lines), and wrote the test suite (test_api.py).

Haresh R (230701097): Built the complete React frontend including the App component (App.jsx – 124 lines), Charts dashboard (Charts.jsx – 204 lines), Chatbot interface (Chatbot.jsx – 201 lines), FileUpload component (FileUpload.jsx – 70 lines), Login system (Login.jsx – 320 lines), and all CSS styling. Also implemented the authentication module (auth.py) and storage module (storage.py).

All members collaborated on integration testing, documentation, project report preparation, and code review. Regular team meetings ensured alignment on architecture decisions and implementation priorities.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

This project successfully delivered FinanceBot, an AI-powered financial data analysis chatbot that comprehensively addresses the core challenges identified in the problem statement: manual analysis overhead, lack of predictive capability, cost barriers to AI tools, absence of conversational interfaces, and fragmented financial workflows.

The platform integrates three powerful analytical capabilities into a unified, accessible application: automated financial analysis with trend computation and anomaly detection, ML-based revenue forecasting with multi-model comparison and confidence intervals, and AI-powered conversational advisory through Google Gemini Pro. These capabilities, previously available only through expensive enterprise platforms or custom data science implementations, are now accessible to SMEs and independent analysts through a simple web interface.

The technical implementation demonstrates proficiency across multiple modern development disciplines. The FastAPI backend provides a high-performance, well-structured API layer with comprehensive error handling and validation. The React frontend delivers an intuitive dashboard experience with real-time data visualization and a natural conversational interface. The multi-stage Docker build ensures efficient, reproducible deployment. The IQR-based anomaly detection, multi-model ML forecasting with automatic selection, and structured AI advisory responses collectively deliver a robust, practical, and extensible financial analysis tool.

From a user perspective, the platform significantly reduces the time and expertise required for financial data analysis. Tasks that previously required hours of manual spreadsheet work, specialized statistical knowledge, or expensive enterprise software can now be accomplished in minutes through simple CSV upload and natural-language questions. The AI advisor provides CFO-level insights with evidence-based reasoning, making sophisticated financial analysis accessible to users without data science backgrounds.

6.2 FUTURE SCOPE

The current implementation provides a solid and extensible foundation for numerous future enhancements. The development roadmap is organized into three phases:

Short-Term Enhancements (3–6 months)

- PostgreSQL or MongoDB integration for persistent data storage, replacing the current in-memory store.
- Support for additional file formats including Excel (.xlsx), PDF financial statements, and JSON data feeds.
- Enhanced authentication with JWT tokens, password reset functionality, and session expiration.
- User dashboard with upload history, saved analyses, and favorite reports.
- Export functionality for analysis results (PDF reports, Excel downloads).

Medium-Term Enhancements (6–12 months)

- Advanced ML models including ARIMA, Prophet, and LSTM neural networks for time-series forecasting.
- Real-time data streaming integration for live financial dashboards.
- Multi-user workspaces with role-based access control (Admin, Analyst, Viewer).
- Scheduled automated analysis with email/notification delivery.
- Custom KPI configuration and alerting based on user-defined thresholds.

Long-Term Vision (12+ months)

- Cloud deployment on Azure Container Apps or AWS ECS with auto-scaling and load balancing.
- Integration with accounting platforms (QuickBooks, Tally, Zoho Books) via API connectors.
- Multi-language support for the AI advisor (Hindi, Tamil, regional languages).
- Progressive Web App (PWA) development for mobile-responsive offline-capable access.

- Advanced visualization capabilities including interactive drill-down reports, customizable dashboards, and comparative analysis views.
- Multi-tenant SaaS architecture for national/international expansion.

REFERENCES

- [1] FastAPI Documentation – Modern, fast web framework for building APIs with Python. Available at: <https://fastapi.tiangolo.com/>
- [2] Google Gemini API Documentation – Generative AI models for content generation. Available at: <https://ai.google.dev/docs>
- [3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. Available at: <https://scikit-learn.org/>
- [4] React Documentation – A JavaScript library for building user interfaces. Available at: <https://react.dev/>
- [5] Docker Multi-Stage Builds – Use multi-stage builds to optimize Dockerfiles. Available at: <https://docs.docker.com/build/building/multi-stage/>
- [6] Recharts – A composable charting library built on React components. Available at: <https://recharts.org/>
- [7] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference. Pandas documentation: <https://pandas.pydata.org/docs/>
- [8] Vite – Next Generation Frontend Tooling. Available at: <https://vitejs.dev/>
- [9] Docker Compose Specification – Defining and running multi-container Docker applications. Available at: <https://docs.docker.com/compose/>
- [10] NumPy – The fundamental package for scientific computing with Python. Available at: <https://numpy.org/>
- [11] Uvicorn – An ASGI web server for Python. Available at: <https://www.uvicorn.org/>
- [12] Google GenAI Python SDK – Official Python client for Google Generative AI. Available at: <https://pypi.org/project/google-genai/>

APPENDICES

Appendix A – Terraform Code Snippets

```

main.tf x
terraform > main.tf
80 resource "azurerm_container_app" "financebot" {
103   template {
106     volume {
107       name = "userdata"
108       storage_type = "AzureFile"
109       storage_name = azurerm_container_app_environment_storage.users_data.name
110     }
111   }
112   container {
113     name = var.app_name
114     image = "${azurerm_container_registry.main.login_server}/${var.app_name}:1
115     cpu = var.cpu
116     memory = var.memory
117   }
118   env {
119     name = "PORT"
120     value = tostring(var.container_port)
121   }
122   env {
123     name = "GEMINI_API_KEY"
124     secret_name = "gemini-api-key"
125   }
126   volume_mounts {
127     name = "userdata"
128     path = "/app/users.json"
129     # Mount the share root; the app writes users.json directly at this path
130     sub_path = "users.json"
131   }
132   liveness_probe {
133     transport = "HTTP"
134     path = "/health"
135     port = var.container_port
136     initial_delay = 10

```

```

main.tf X
terraform > main.tf
78 # --- Container App ---
79
80 resource "azurerm_container_app" "financebot" {
81   name = var.app_name
82   container_app_environment_id = azurerm_container_app_environment.main.id
83   resource_group_name = azurerm_resource_group.main.name
84   revision_mode = "Single"
85   tags = local.tags
86
87   registry {
88     server = azurerm_container_registry.main.login_server
89     username = azurerm_container_registry.main.admin_username
90     password_secret_name = "acr-password"
91   }
92
93   secret {
94     name = "acr-password"
95     value = azurerm_container_registry.main.admin_password
96   }
97
98   secret {
99     name = "gemini-api-key"
100    value = var.gemini_api_key
101  }
102
103  template {
104    min_replicas = var.min_replicas
105    max_replicas = var.max_replicas
106
107    volume {
108      name = "userdata"
109      storage_type = "AzureFile"
110      storage_name = azurerm_container_app_environment_storage.users_data.name
111    }
112
113    container {
114      name = var.app_name
115      image = "${azurerm_container_registry.main.login_server}/${var.app_name}:1

```

```

main.tf X
terraform > main.tf
40
41 # --- Storage Account for persistent users.json ---
42
43 resource "azurerm_storage_account" "main" {
44   name = "${local.acr_name}storage"
45   resource_group_name = azurerm_resource_group.main.name
46   location = azurerm_resource_group.main.location
47   account_tier = "Standard"
48   account_replication_type = "LRS"
49   tags = local.tags
50 }
51
52 resource "azurerm_storage_share" "users_data" {
53   name = "userdata"
54   storage_account_name = azurerm_storage_account.main.name
55   quota = 1 # GB
56 }
57
58 # --- Container Apps Environment ---
59
60 resource "azurerm_container_app_environment" "main" {
61   name = "${var.app_name}-env"
62   resource_group_name = azurerm_resource_group.main.name
63   location = azurerm_resource_group.main.location
64   log_analytics_workspace_id = azurerm_log_analytics_workspace.main.id
65   tags = local.tags
66 }
67
68 # Mount the Azure File Share into the environment so Container Apps can use it
69 resource "azurerm_container_app_environment_storage" "users_data" {
70   name = "userdata"
71   container_app_environment_id = azurerm_container_app_environment.main.id
72   account_name = azurerm_storage_account.main.name
73   share_name = azurerm_storage_share.users_data.name
74   access_key = azurerm_storage_account.main.primary_access_key
75   access_mode = "ReadWrite"
76 }

```

```

main.tf X
terraform > main.tf
1  locals {
2    tags = {
3      project    = var.app_name
4      environment = "production"
5      managed_by = "terraform"
6    }
7    # Sanitised name – ACR only allows alphanumeric
8    acr_name = replace(var.app_name, "-", "")
9  }
10
11 # — Resource Group —
12
13 resource "azurerm_resource_group" "main" {
14   name     = var.resource_group_name
15   location = var.location
16   tags     = local.tags
17 }
18
19 # — Container Registry —
20
21 resource "azurerm_container_registry" "main" {
22   name                = "${local.acr_name}registry"
23   resource_group_name = azurerm_resource_group.main.name
24   location             = azurerm_resource_group.main.location
25   sku                 = "Basic"
26   admin_enabled       = true
27   tags                = local.tags
28 }
29
30 # — Log Analytics (required by Container Apps Environment) —
31
32 resource "azurerm_log_analytics_workspace" "main" {
33   name                = "${var.app_name}-logs"
34   resource_group_name = azurerm_resource_group.main.name
35   location             = azurerm_resource_group.main.location
36   sku                 = "PerGB2018"
37   retention_in_days   = 30
38   tags                = local.tags
39 }

outputs.tf X
terraform > outputs.tf
1  output "app_url" {
2    description = "Public URL of the FinanceBot"
3    value       = "https://${azurerm_container_app.financebot.ingress[0].fqdn}"
4  }
5
6  output "acr_login_server" {
7    description = "ACR login server – use this for docker push"
8    value       = azurerm_container_registry.main.login_server
9  }
10
11 output "acr_push_commands" {
12   description = "Commands to build and push the image"
13   value = <<-EOT
14     az acr login --name ${azurerm_container_registry.main.name}
15     docker build -t ${azurerm_container_registry.main.login_server}/${var.app_name}
16     docker push ${azurerm_container_registry.main.login_server}/${var.app_name}:1
17   EOT
18 }
19
20 output "resource_group" {
21   value = azurerm_resource_group.main.name
22 }
23

```

```

variables.tf X
terraform > variables.tf
1  variable "resource_group_name" {
2      description = "Name of the Azure resource group"
3      type       = string
4      default    = "financebot-rg"
5  }
6
7  variable "location" {
8      description = "Azure region"
9      type       = string
10     default    = "eastus"
11 }
12
13 variable "app_name" {
14     description = "Base name for all resources"
15     type       = string
16     default    = "financebot"
17 }
18
19 variable "gemini_api_key" {
20     description = "Google Gemini API key"
21     type       = string
22     sensitive  = true
23 }
24
25 variable "container_port" {
26     description = "Port the container listens on"
27     type       = number
28     default    = 8000
29 }
30
31 variable "cpu" {
32     description = "vCPU allocation for the container (0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0)"
33     type       = number
34     default    = 0.5
35 }
36
37 variable "memory" {
38     description = "Memory allocation (must pair with cpu - see Azure Container Apps"

```

```

variables.tf X
terraform > variables.tf
1  variable "resource_group_name" {
2      description = "Name of the Azure resource group"
3      type       = string
4      default    = "financebot-rg"
5  }
6
7  variable "location" {
8      description = "Azure region"
9      type       = string
10     default    = "eastus"
11 }
12
13 variable "app_name" {
14     description = "Base name for all resources"
15     type       = string
16     default    = "financebot"
17 }
18
19 variable "gemini_api_key" {
20     description = "Google Gemini API key"
21     type       = string
22     sensitive  = true
23 }
24
25 variable "container_port" {
26     description = "Port the container listens on"
27     type       = number
28     default    = 8000
29 }
30
31 variable "cpu" {
32     description = "vCPU allocation for the container (0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0)"
33     type       = number
34     default    = 0.5
35 }
36
37 variable "memory" {
38     description = "Memory allocation (must pair with cpu - see Azure Container Apps"
39     type       = string

```

Appendix B – Pipeline YAMLs

```

github > workflows > ! deploy.yml
1  name: Build & Deploy FinanceBot
2
3  on:
4    push:
5      branches: [main]
6    pull_request:
7      branches: [main]
8    workflow_dispatch:
9
10 env:
11   APP_NAME: financebot
12   CONTAINER_PORT: 8000
13
14 jobs:
15   # — Test —
16   test:
17     name: Run Tests
18     runs-on: ubuntu-latest
19
20     steps:
21       - uses: actions/checkout@v4
22
23       - name: Set up Python
24         uses: actions/setup-python@v5
25         with:
26           python-version: "3.11"
27           cache: pip
28
29       - name: Install dependencies
30         run: pip install -r requirements.txt
31
32       - name: Run tests
33         run: python -m pytest tests/ -v --tb=short
34         env:
35           GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
36
37   # — Build & Push Image —
38   build:
39     name: Build & Push to ACR

```

```

! prod.yml ×
github > workflows > ! prod.yml
1  name: Production Deploy – FinanceBot
2
3  on:
4    # Manually trigger with optional image tag; defaults to latest
5    workflow_dispatch:
6      inputs:
7        image_tag:
8          description: "Image tag to deploy (leave blank for latest)"
9          required: false
10         default: "latest"
11        canary_weight:
12          description: "% traffic to new revision before full cutover (0 = skip car
13          required: false
14          default: "20"
15
16    # Or auto-trigger after the CI deploy workflow succeeds on main
17    workflow_run:
18      workflows: ["Build & Deploy FinanceBot"]
19      types: [completed]
20      branches: [main]
21
22 env:
23   APP_NAME: financebot
24   RESOURCE_GROUP: ${ secrets.RESOURCE_GROUP }
25   ACR_LOGIN_SERVER: ${ secrets.ACR_LOGIN_SERVER }
26
27 jobs:
28   # — Pre-flight checks —
29   preflight:
30     name: Pre-flight Checks
31     runs-on: ubuntu-latest
32     # Skip if triggered by a failed workflow_run
33     if: >
34       github.event_name == 'workflow_dispatch' ||
35       github.event.workflow_run.conclusion == 'success'
36
37     outputs:
38       image_tag: ${ steps.resolve.outputs.image_tag }
39       current_revision: ${ steps.revision.outputs.current_revision }

```

Appendix C – Screenshots (Deployments, AI Integration, Security Scans)

Successful ci/cd run:

The screenshot displays a GitHub Actions workflow run for the file `main-deploy.yml`. The workflow was triggered by a push to the `main` branch by user `haresh-r2103` (commit `456e52d`) and has a status of **Success** with a total duration of **45s**. The workflow consists of two jobs: `build-and-deploy-backend` (25s) and `build-and-deploy-frontend` (13s), both of which completed successfully. The interface includes a sidebar with navigation options: Summary, All jobs, Run details, Usage, and Workflow file. The 'Annotations' section shows two warnings regarding deprecated Node.js 20 actions. The footer contains the GitHub logo, copyright information for 2026, and various links including Terms, Privacy, Security, Status, Community, Docs, Contact, Manage cookies, and a privacy notice.

Triggered via push 6 minutes ago
haresh-r2103 pushed `456e52d` `main` **Success** Total duration: **45s** Artifacts: -

main-deploy.yml
on: push

build-and-deploy-backend 25s → build-and-deploy-frontend 13s

Annotations
2 warnings

build-and-deploy-backend
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-python@v5. [Show more](#)

build-and-deploy-frontend
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-node@v4. Acti... [Show more](#)

© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) Do not share my personal information